Defesa Cibernética & Cloud
Prof. Eduardo Popovici

2TDCPR

API de Tarefas

API REST com Flask e banco SQLite, publicada de ponta a ponta em uma máquina virtual na Azure.

`POST /tarefas`

Cliente
curl ou navegador

HTTP

Rede
IP público 20.104.255.148 e NSG

portas 80 e 22

Máquina virtual
vm-gs, Ubuntu 22.04, gunicorn e Flask

Standard_B2ats_v2

Dados
SQLite no disco Premium SSD da VM

`/var/lib/api-tarefas/tarefas.db`

Aluno
Victor Marcolongo Pereira

RM
566703

Data
02/11/2026

API
http://20.104.255.148

Repositório
github.com/vlongo404/GS-cloud

Sumário

1	Objetivo e itens da atividade	2
2	Visão geral e arquitetura	3
3	Linguagem de programação e tecnologias	4
4	A API	5
5	Elementos de nuvem utilizados	7
6	Banco de dados SQLite	9
7	Deploy passo a passo	10
8	Evidências (prints de tela)	12
9	Testes	17
10	Segurança, limitações e melhorias	18
11	Repositório e referências	18
Anexo A	Código-fonte completo: app.py	19
Anexo B	Arquivos de deploy	23

1 Objetivo e itens da atividade

A atividade consiste em desenvolver uma API básica com banco de dados SQLite, hospedá-la de ponta a ponta na nuvem, compartilhar o código no GitHub e entregar a documentação completa, com esquemas visuais, em PDF. O professor pediu quatro itens; a tabela indica onde cada um está neste documento.

Linguagem	Banco de dados	Máquina virtual	Armazenamento
Python 3 e Flask	**SQLite**	**Standard_B2ats_v2**	**Premium SSD 30 GiB**

Item exigido	Onde está neste documento
1. Prints de tela	Seção 8 (Evidências), com marca d'água de autoria
2. Explicação detalhada	Seções 2 a 7, 9 e 10, mais os anexos com o código completo
3. Link do repositório no Git	Capa e seção 11: https://github.com/vlongo404/GS-cloud
4. Elementos de nuvem: armazenamento, VM (SKU) e linguagem de programação	Seção 5 (e seção 3 para a linguagem)

2 Visão geral e arquitetura

A solução é uma API REST de tarefas (cadastro, consulta, atualização e remoção) escrita em Python com Flask. Os dados ficam em um arquivo SQLite dentro de uma máquina virtual Linux na Azure (Canada Central). O gunicorn serve a aplicação na porta 80, o systemd mantém o serviço no ar, e uma regra no grupo de segurança de rede (NSG) libera o acesso HTTP pela internet.

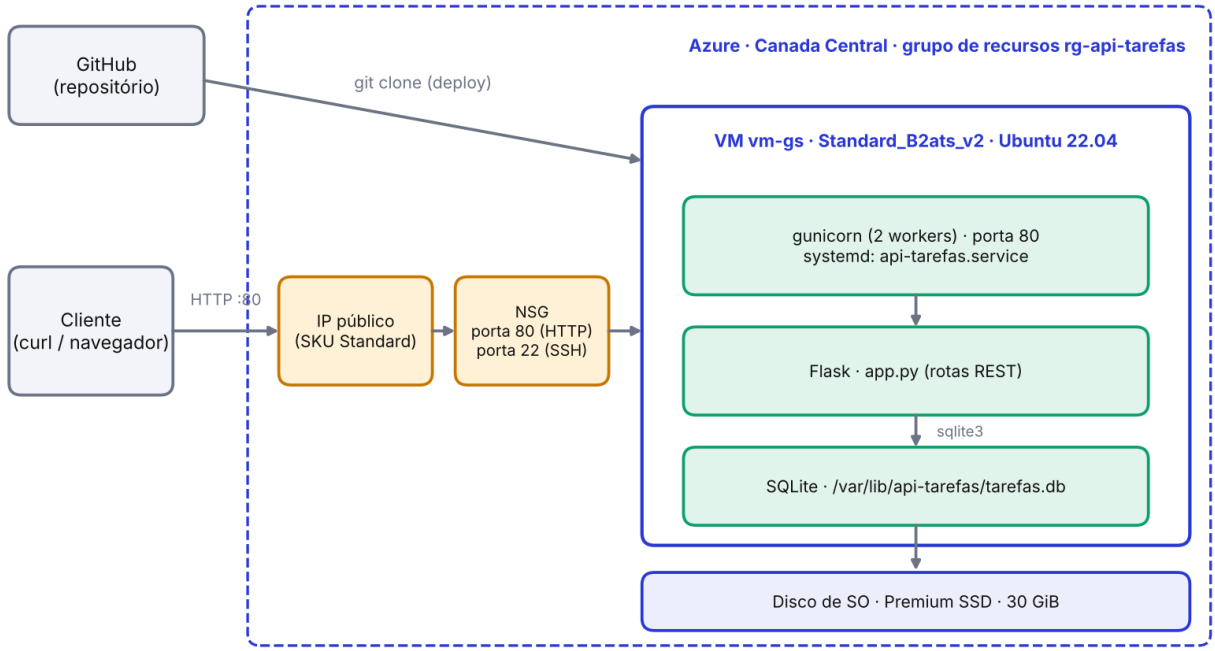


Figura 1 Arquitetura da solução na Azure.

Caminho de uma requisição:

- O cliente (curl ou navegador) envia a requisição HTTP para o IP público da VM, porta 80.
- O NSG avalia as regras de entrada e permite o tráfego da porta 80.

- O gunicorn recebe a conexão e entrega a requisição a um dos 2 workers, que executam o Flask.
- O Flask valida os dados, consulta ou grava no SQLite usando consultas parametrizadas e devolve a resposta em JSON.

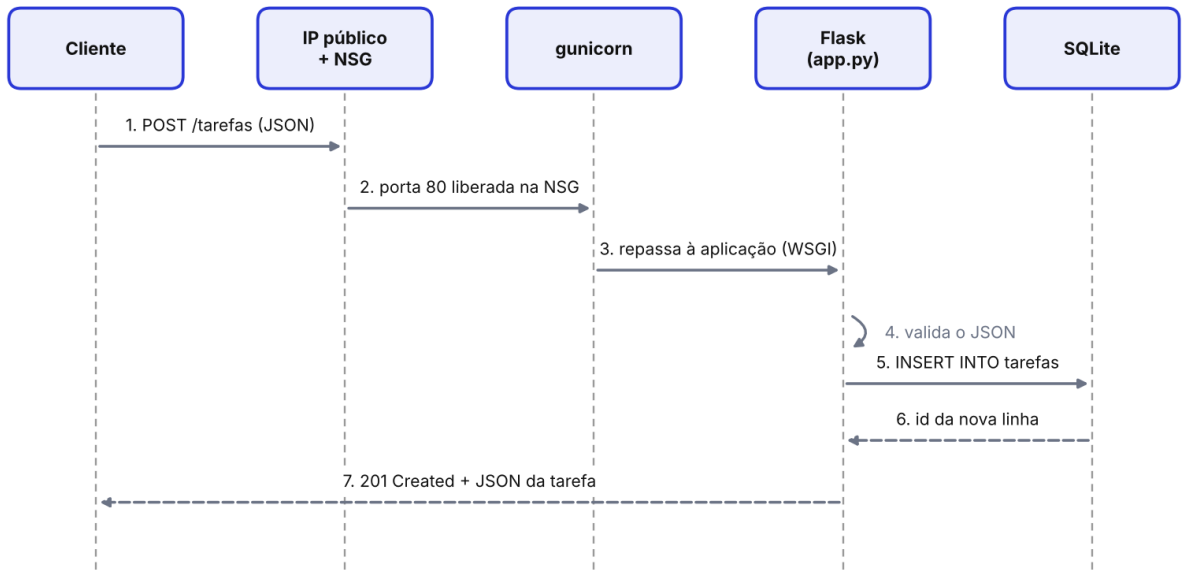


Figura 2 Fluxo de uma requisição POST /tarefas, do cliente até o banco.

3 Linguagem de programação e tecnologias

A linguagem usada é **Python 3**, com o micro-framework **Flask**. Na VM, o Ubuntu 22.04 traz o Python 3.10; os testes locais foram executados em Python 3.13. A escolha prioriza código curto, fácil de explicar e sem dependências além do necessário.

Tecnologia	Função no projeto	Por que foi escolhida
Python 3	Linguagem da API	Sintaxe simples e módulo sqlite3 na biblioteca padrão.
Flask	Framework web (rotas, JSON, erros)	Mínimo e direto: uma rota é uma função com um decorador.
SQLite (módulo sqlite3)	Banco de dados relacional em arquivo único	Sem servidor de banco para instalar ou administrar; exigido pela atividade.
gunicorn	Servidor WSGI de produção	O servidor embutido do Flask é só para desenvolvimento.
systemd	Gerenciador de serviços do Linux	Inicia a API no boot e reinicia automaticamente se o processo cair.

Tecnologia	Função no projeto	Por que foi escolhida
Ubuntu Server 22.04 LTS	Sistema operacional da VM	Imagem oficial, suporte de longo prazo, apt para pacotes.
Git e GitHub	Versionamento e entrega do código	A VM clona o repositório para publicar a versão.
Azure CLI	Criação da infraestrutura por comandos	Processo repetível e documentado (infraestrutura como script).

4 A API

4.1 Rotas

Método	Rota	Descrição	Resposta
`GET`	`/`	Informações e lista de rotas	200
`GET`	`/health`	Verifica a API e o acesso ao banco	200
`GET`	`/tarefas`	Lista as tarefas (filtro opcional ?concluida=0 ou 1)	200
`GET`	`/tarefas/<id>`	Busca uma tarefa pelo id	200 ou 404
`POST`	`/tarefas`	Cria uma tarefa (campo titulo obrigatório)	201 ou 400
`PUT`	`/tarefas/<id>`	Atualiza titulo, descricao e/ou concluida	200, 400 ou 404
`DELETE`	`/tarefas/<id>`	Remove a tarefa	204 ou 404

4.2 Exemplos de uso

Os comandos abaixo foram executados em teste local com o gunicorn. Na VM, troque `127.0.0.1:5000` pelo IP público (porta 80).

terminal

```
$ curl -X POST http://127.0.0.1:5000/tarefas -H 'Content-Type: application/json' \
       -d '{"titulo":"Estudar Azure","descricao":"VM e storage"}'
HTTP 201
{
  "atualizada_em": "2026-11-02T16:58:09+00:00",
  "concluida": false,
  "criada_em": "2026-11-02T16:58:09+00:00",
  "descricao": "VM e storage",
  "id": 1,
  "titulo": "Estudar Azure"
}

$ curl -X PUT http://127.0.0.1:5000/tarefas/1 -H 'Content-Type: application/json' \
       -d '{"concluida":true}'
HTTP 200   (mesma tarefa, com "concluida": true)

$ curl -X DELETE http://127.0.0.1:5000/tarefas/1
HTTP 204   (sem corpo)

$ curl http://127.0.0.1:5000/tarefas/1
HTTP 404
{"erro": "tarefa nao encontrada"}
```

4.3 Validação e erros

Situação	Resposta
Corpo ausente, que não seja JSON ou que não seja um objeto	400 com {"erro": "..."}
titulo ausente (POST), vazio ou com mais de 120 caracteres	400
concluida diferente de true ou false; descricao que não seja texto	400
Id inexistente em GET, PUT ou DELETE	404
Rota inexistente ou método não permitido	404 ou 405, também em JSON

4.4 Cuidados de segurança no código

- **Consultas parametrizadas**: os valores sempre vão como parâmetros (?), nunca concatenados no SQL. Um teste automatizado grava um título com tentativa de SQL injection e confirma que a tabela continua intacta.
- **Validação de entrada** antes de tocar no banco (tipo, tamanho e campos aceitos).
- **Nomes de coluna no UPDATE** vêm do dicionário já validado, nunca direto do cliente.
- **Modo debug desligado**: erros internos não expõem código nem variáveis ao cliente.
- **Serviço sem root**: o gunicorn roda como usuário comum, com permissão apenas para ouvir na porta 80.

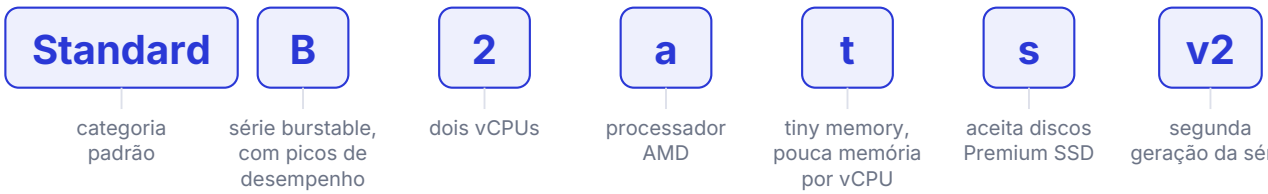
5 Elementos de nuvem utilizados

5.1 Resumo dos recursos

Recurso Azure	Configuração usada	Função
Grupo de recursos	rg-api-tarefas (metadados em Brazil South; recursos em Canada Central)	Agrupa todos os recursos; apagar o grupo remove tudo.
Máquina virtual	vm-gs, SKU Standard_B2ats_v2, 2 vCPU, 1 GiB de RAM, Ubuntu Server 22.04 LTS (Canonical, x64 Gen2)	Executa o gunicorn, o Flask e o SQLite.
Armazenamento (disco gerenciado do SO)	Premium SSD (Premium_LRS), 30 GiB	Guarda o sistema, o código, o ambiente Python e o arquivo do banco.
IP público	SKU Standard (endereço estático)	Endereço de acesso à API pela internet.
NSG (grupo de segurança de rede)	Entrada liberada: 22/TCP (SSH) e 80/TCP (HTTP)	Firewall da interface de rede da VM.
Rede virtual, sub-rede e NIC	Criadas automaticamente junto com a VM	Conectividade da VM.

5.2 Máquina virtual e SKU

SKU é o código do produto que define o tamanho da máquina. O usado aqui, **Standard_B2ats_v2**, se lê assim:



Esse tamanho oferece 2 vCPUs e 1 GiB de memória.

> **Por que Canada Central e não Brazil South**
> O plano inicial era o Standard_B1s em Brazil South, mas a assinatura usada retornou o erro SkuNotAvailable (restrição de capacidade) para ele nessa região. Por isso a VM foi criada em **Canada Central**, com o Standard_B2ats_v2. O grupo de recursos continua com os metadados em Brazil South, o que não afeta o funcionamento.

A série B acumula créditos de CPU enquanto a VM trabalha abaixo do seu desempenho base e pode usar até 100% da vCPU em momentos de pico. Isso combina com uma API de poucas requisições, que fica ociosa na maior parte do tempo, e é a opção de menor custo para a atividade.

Segundo a página de serviços gratuitos da Microsoft, contas novas da Azure recebem 750 horas por mês de cada um dos tamanhos B1s, B2pts v2 e B2ats v2, por 12 meses (oferta

para contas elegíveis). O tamanho usado aqui, B2ats v2, está nessa lista.

5.3 Armazenamento

- **Disco do sistema operacional**: disco gerenciado Premium SSD (Premium_LRS), 30 GiB. O tipo Premium SSD foi o padrão sugerido pelo portal para este tamanho (a letra *s* do SKU indica suporte a discos Premium). A redundância é local (LRS), com três cópias dentro do mesmo datacenter. Para reduzir custo seria possível trocar para Standard SSD, que também atenderia a uma carga leve como esta.
- **Onde fica o banco**: o arquivo `/var/lib/api-tarefas/tarefas.db` está nesse disco persistente. A VM também costuma ter um disco temporário local, que **não** é persistente e por isso não é usado para o banco.
- **Por que não usar Storage Account ou Azure SQL**: o SQLite é um arquivo local e precisa de acesso rápido a disco; ele não funciona como serviço de banco remoto. A atividade pede SQLite, então o armazenamento adequado é o disco da própria VM.
- **Cópia de segurança (melhoria)**: tirar um snapshot do disco ou copiar o arquivo .db para um contêiner de Blob Storage com agendamento.

5.4 Rede e segurança

Porta	Protocolo	Origem	Como foi criada	Motivo
22	TCP (SSH)	Qualquer	Marcada na criação da VM	Administração remota da VM por SSH.
80	TCP (HTTP)	Qualquer	Marcada na criação da VM	Acesso público à API.
Demais	-	-	Regras padrão do NSG	Entrada negada.

Em produção, a porta 22 deveria aceitar apenas o IP do administrador, e o tráfego HTTP deveria passar por HTTPS (seção 10).

5.5 Custos e limpeza

> **Apague tudo depois da avaliação**
> Uma VM desalocada deixa de cobrar computação, mas o disco e o IP público continuam sendo cobrados. Para encerrar tudo, o comando abaixo remove o grupo de recursos inteiro.

Azure CLI

```
az group delete --name rg-api-tarefas --yes --no-wait
```

6 Banco de dados SQLite

O SQLite é um banco relacional embutido: todo o banco é um único arquivo, acessado pelo módulo sqlite3 do Python, sem processo de servidor. A tabela é criada automaticamente na primeira execução da aplicação (CREATE TABLE IF NOT EXISTS). A evidência 11 mostra o arquivo do banco na VM e a tarefa gravada nele.

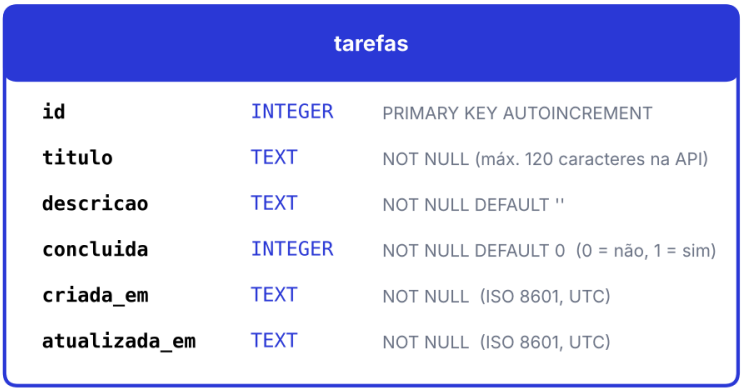


Figura 3 Modelo de dados: tabela tarefas.

```sql
CREATE TABLE IF NOT EXISTS tarefas (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    titulo        TEXT    NOT NULL,
    descricao     TEXT    NOT NULL DEFAULT '',
    concluida     INTEGER NOT NULL DEFAULT 0,
    criada_em     TEXT    NOT NULL,
    atualizada_em TEXT    NOT NULL
);
```

- **Modo WAL** (PRAGMA journal_mode=WAL): permite que leituras continuem enquanto ocorre uma gravação, o que ajuda com os 2 workers do gunicorn.
- **Limitação conhecida**: o SQLite aceita uma gravação por vez. Para a carga desta atividade é suficiente; para muitos acessos simultâneos seria necessário migrar para PostgreSQL ou Azure SQL.
- **Datas** em texto ISO 8601, em UTC.

7 Deploy passo a passo

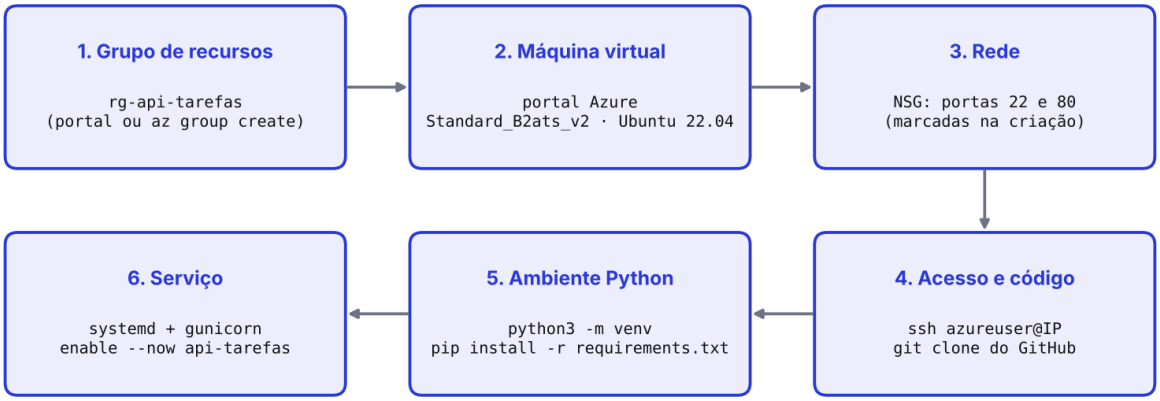


Figura 4 Sequência do deploy, da criação da VM ao serviço no ar.

7.1 Criação da VM no portal da Azure

Campo	Valor usado
Grupo de recursos	rg-api-tarefas
Nome da máquina virtual	vm-gs
Região	Canada Central
Imagem	Ubuntu Server 22.04 LTS (Canonical, x64 Gen2)
Tamanho (SKU)	Standard_B2ats_v2 (2 vCPUs, 1 GiB)
Usuário administrador	azureuser (mesmo usuário definido no serviço systemd)
Autenticação SSH	Senha
Portas de entrada públicas	SSH (22) e HTTP (80)
Disco do sistema operacional	Premium SSD (Premium_LRS), 30 GiB

7.1.1 Equivalente por linha de comando (deploy/azure-cli.sh)

```
Azure CLI

az group create --name rg-api-tarefas --location canadacentral
az vm create --resource-group rg-api-tarefas --name vm-gs \
   --image Ubuntu2204 --size Standard_B2ats_v2 --admin-username azureuser --generate-ssh-keys \
   --storage-sku Premium_LRS --os-disk-size-gb 30 --public-ip-sku Standard
az vm open-port --resource-group rg-api-tarefas --name vm-gs --port 80 --priority 900
az vm show -d --resource-group rg-api-tarefas --name vm-gs --query publicIps -o tsv
```

7.2 Configuração da VM (deploy/setup-vm.sh)

Depois de conectar com `ssh azureuser@IP_PUBLICO`, o script instala python3-venv e git, clona o repositório em /opt/api-tarefas, cria o ambiente virtual, instala as dependências do requirements.txt, registra o serviço no systemd e testa o /health.

```
dentro da VM

./setup-vm.sh https://github.com/vlongo404/GS-cloud.git
```

7.3 Serviço systemd (deploy/api-tarefas.service)

Diretiva	Efeito
User=azureuser	O processo roda sem privilégios de root.
StateDirectory=api-tarefas	O systemd cria /var/lib/api-tarefas, pertencente ao usuário, para o arquivo .db.
Environment=DB_PATH=...	Informa à aplicação onde está o banco.
ExecStart=... gunicorn --workers 2 --bind 0.0.0.0:80 app:app	Sobe 2 workers e ouve na porta 80.
AmbientCapabilities=CAP_NET_BIND_SERVICE	Permite ouvir em porta menor que 1024 sem ser root.
NoNewPrivileges=true, PrivateTmp=true	Reduzem a superfície de ataque do processo.
Restart=always	Reinicia o serviço automaticamente em caso de falha.
WantedBy=multi-user.target (+ enable)	Inicia o serviço no boot da VM.

7.4 Atualizar a versão publicada

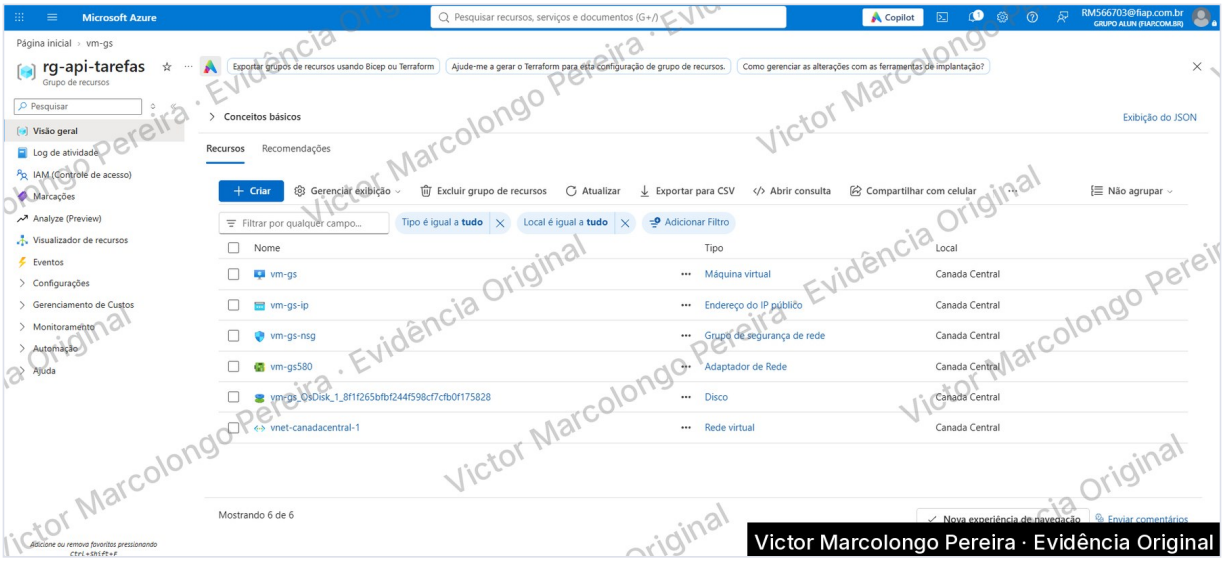
```
dentro da VM

cd /opt/api-tarefas && git pull
.venv/bin/pip install -r requirements.txt
sudo systemctl restart api-tarefas
```

8 Evidências (prints de tela)

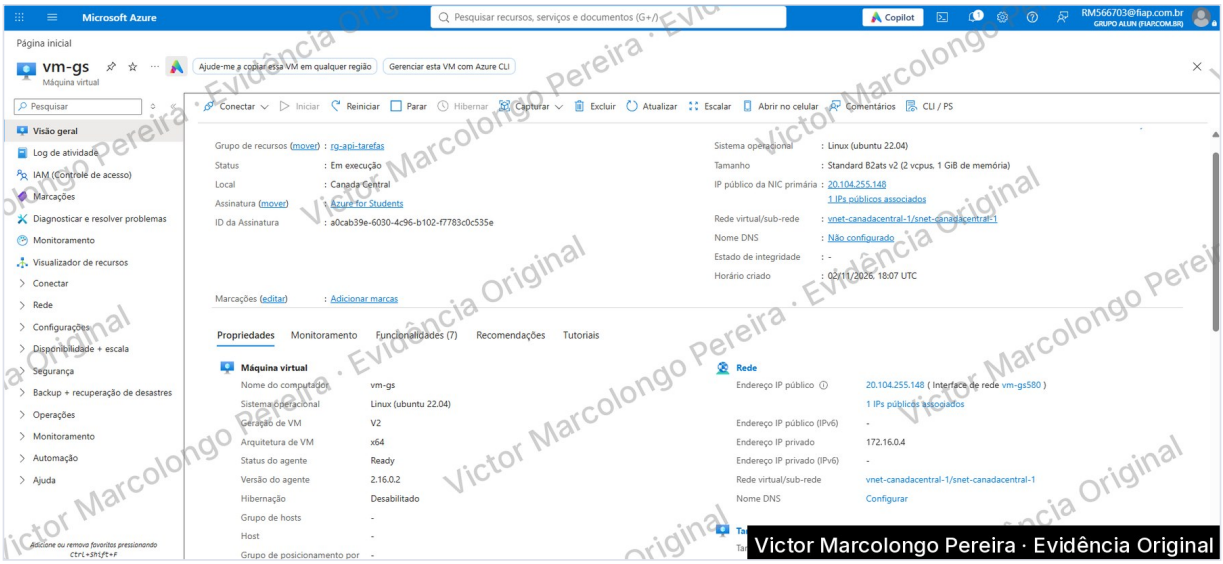
Cada print abaixo recebe, automaticamente, a marca d'água com o nome do aluno e a expressão "Evidência Original". Os arquivos originais ficam intactos em docs/prints.

01 Grupo de recursos na Azure



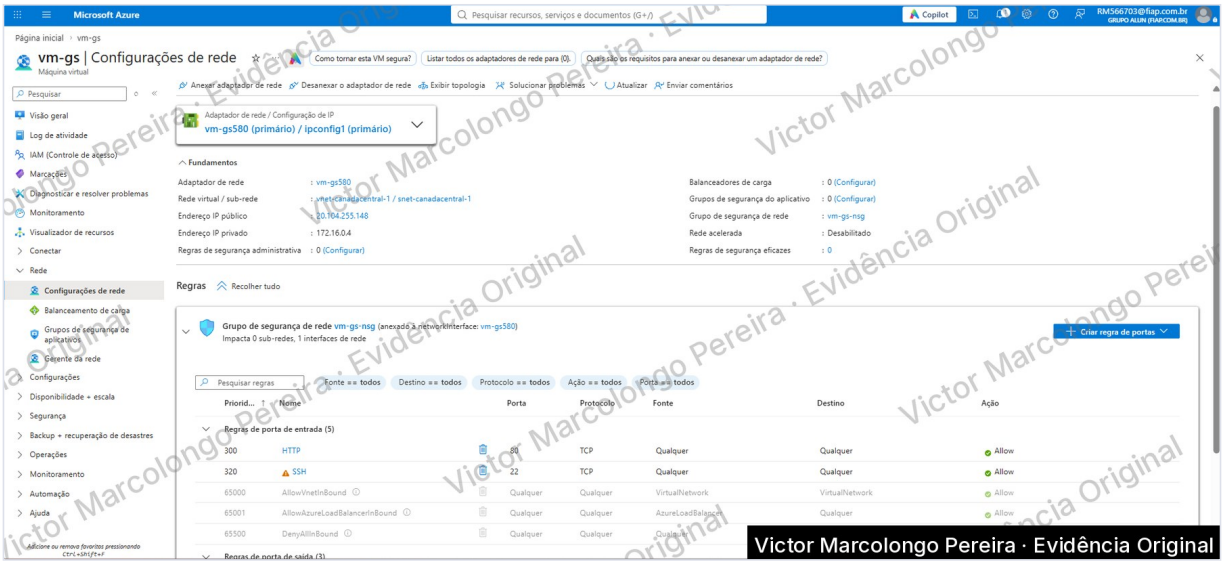
O que comprova. Todos os recursos da solução estão no grupo rg-api-tarefas: VM, disco, IP público, NSG, interface e rede virtual.

02 Visão geral da máquina virtual



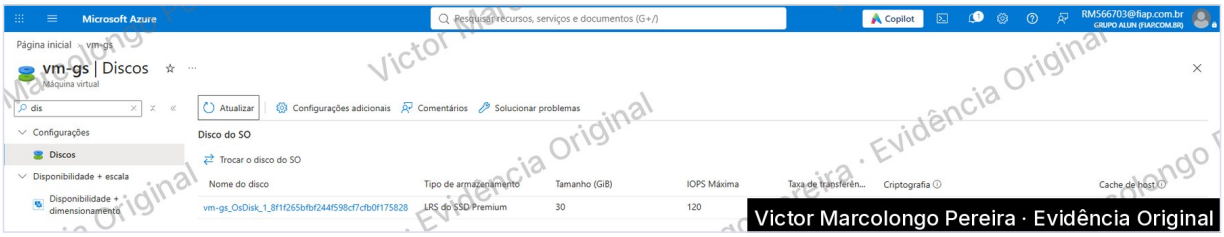
O que comprova. A VM vm-gs está em execução em Canada Central, tamanho Standard_B2ats_v2, Ubuntu 22.04, com o IP público 20.104.255.148.

03 Regras de entrada do NSG



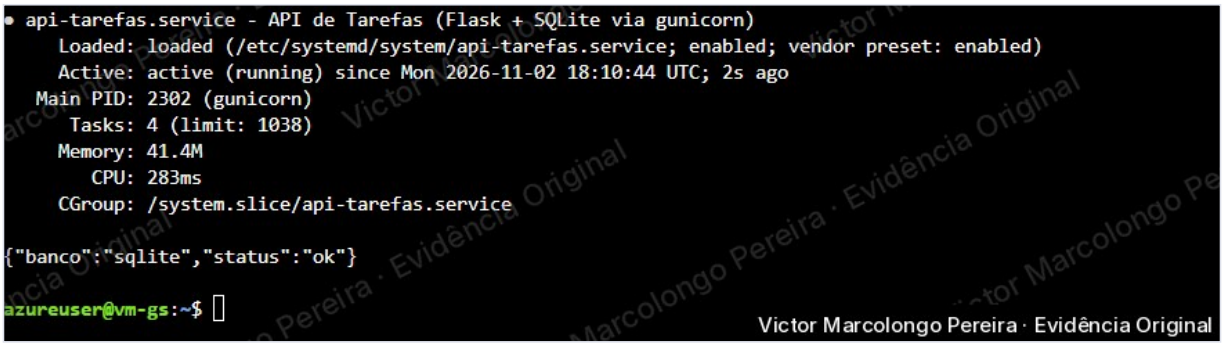
O que comprova. O firewall libera a entrada nas portas 80 (HTTP) e 22 (SSH). Todo o resto segue a regra padrão de negar.

04 Disco do sistema operacional



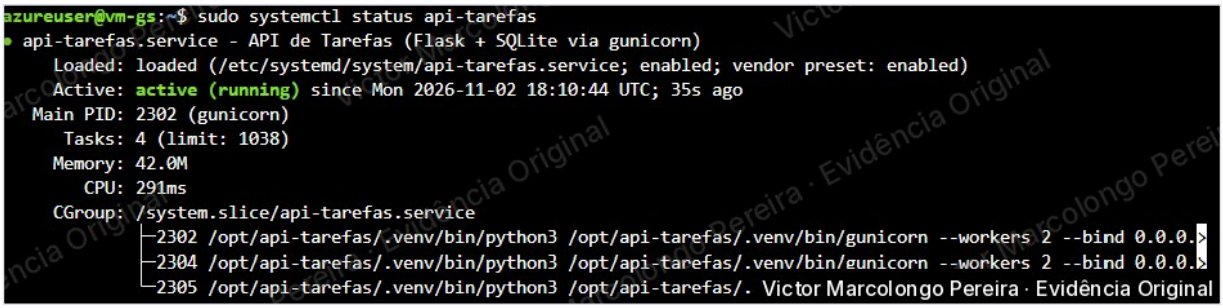
O que comprova. O disco do SO é Premium SSD (LRS) de 30 GiB. Nele ficam o sistema, o código e o arquivo do banco.

05 Configuração da VM via SSH



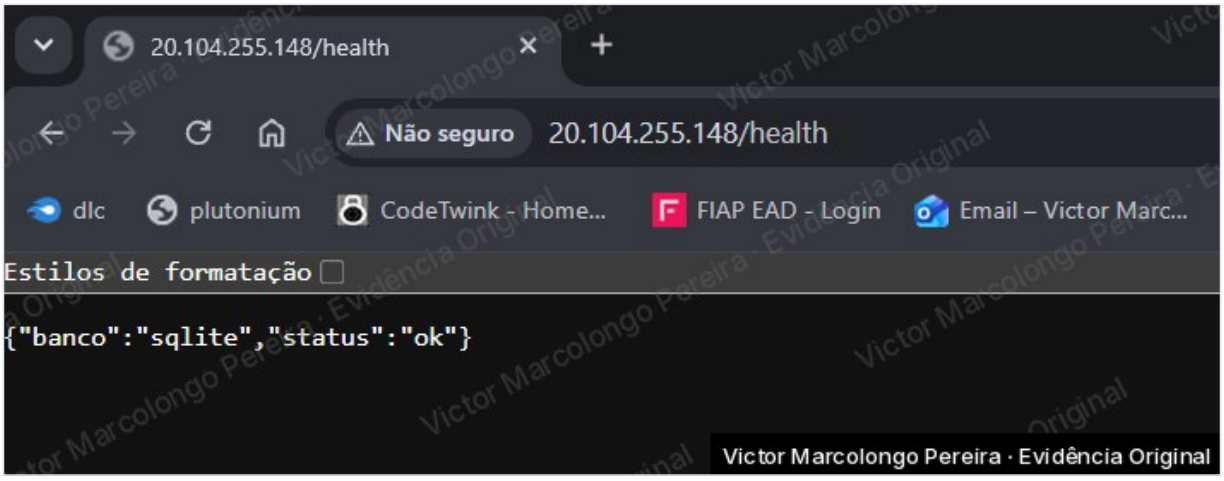
O que comprova. O script setup-vm.sh instalou a aplicação dentro da VM e o /health respondeu {"banco":"sqlite","status":"ok"}.

06 Serviço ativo no systemd



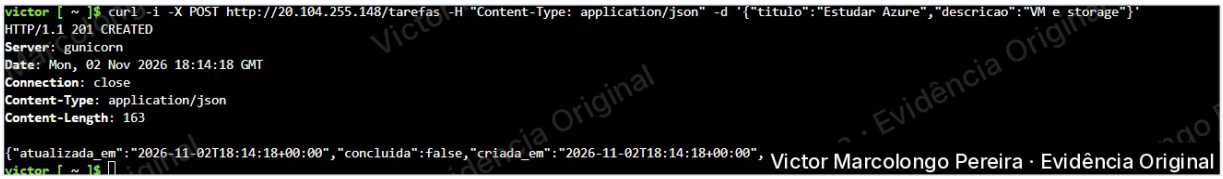
O que comprova. O api-tarefas.service está active (running), com o gunicorn e seus 2 workers.

07 Teste da rota /health pela internet



O que comprova. A API responde fora da VM: o /health abriu no navegador do aluno, pelo IP público.

08 Criando uma tarefa (POST)



O que comprova. POST /tarefas devolve 201 Created com o JSON da tarefa gravada no SQLite (id 1).

09 Listando tarefas (GET)



O que comprova. GET /tarefas devolve a tarefa recém-gravada, o que confirma a persistência no banco.

10 Atualizando e removendo (PUT e DELETE)



O que comprova. O PUT altera concluida para true, o DELETE responde 204 e a consulta seguinte retorna 404.

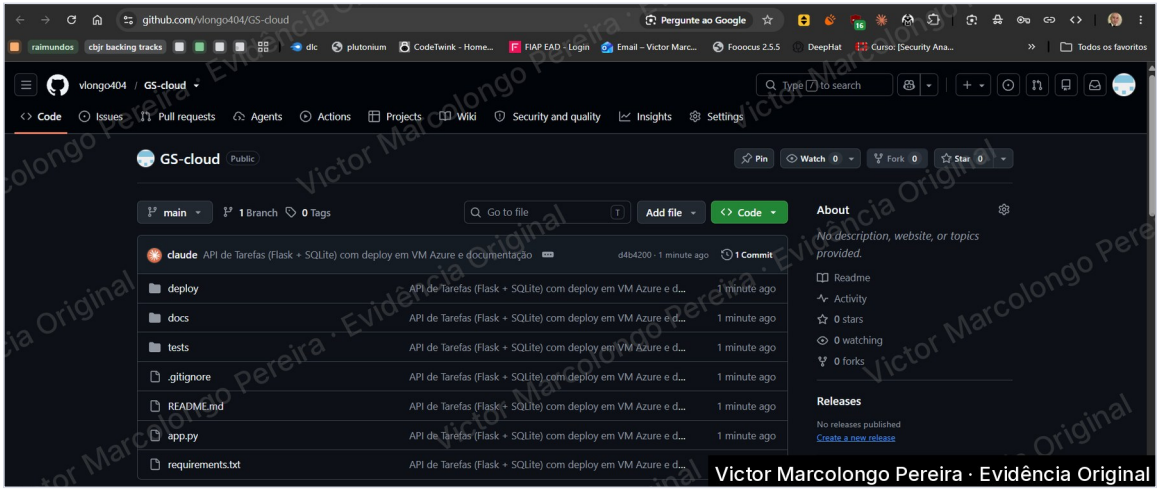
11 Banco SQLite na VM



O que comprova. O arquivo tarefas.db existe em /var/lib/api-tarefas e a tabela tarefas guarda a tarefa criada pela API (consulta direta com sqlite3).

12 Repositório no GitHub





O que comprova. O código está em um repositório público do GitHub, com as pastas deploy, docs e tests.

9 Testes

9.1 Testes automatizados

O arquivo tests/test_api.py usa unittest e o cliente de testes do Flask, com banco temporário. Os 5 testes passaram na execução local (python -m unittest discover -s tests -v).

Teste	O que verifica
`test_health`	A rota /health responde 200 e acessa o banco.
`test_crud_completo`	Fluxo POST, GET, PUT (concluida), filtro ?concluida=1, DELETE e GET final com 404.
`test_validacao`	400 para corpo vazio, título em branco, título com 121 caracteres e corpo que não é JSON.
`test_sql_injection_nao_funciona`	Um título com SQL malicioso é gravado como texto e a tabela continua existindo.
`test_404`	Rota inexistente e PUT em id inexistente retornam 404.

9.2 Teste na nuvem

Depois do deploy, os testes manuais são feitos de fora da VM, apontando para o IP público (`http://20.104.255.148`). Os resultados estão nas evidências 7 a 11 da seção 8.

10 Segurança, limitações e melhorias

Ponto	Situação atual	Melhoria sugerida
Tráfego HTTP	Sem criptografia (porta 80)	Colocar nginx como proxy reverso com certificado Let's Encrypt (HTTPS).
Acesso SSH	Porta 22 aberta para a internet, com login do usuário azureuser	Restringir a origem ao IP do administrador no NSG, usar chave SSH em vez de senha ou usar o Azure Bastion.
Autenticação da API	Rotas públicas, sem login	Chave de API ou token (JWT) nas rotas de escrita.
Banco SQLite	Uma gravação por vez; arquivo em um único disco	Backup por snapshot ou cópia para Blob; migrar para PostgreSQL/Azure SQL se crescer.
Disponibilidade	Uma única VM	Zonas de disponibilidade e balanceador, o que exigiria banco em serviço gerenciado.
Entrega do código	Deploy manual com git pull	Pipeline de CI/CD (GitHub Actions) com testes automáticos antes de publicar.

11 Repositório e referências

Repositório no GitHub: **https://github.com/vlongo404/GS-cloud**

estrutura do repositório

```
GS-cloud/
  app.py                 API Flask + SQLite
  requirements.txt       Flask e gunicorn
  README.md              Resumo, rotas e como rodar
  tests/test_api.py      Testes automatizados
  docs/                  Scripts, diagramas e fontes que geram este PDF
  deploy/                Scripts de infraestrutura e o serviço systemd
```

Referências

- Microsoft Learn - Série Bv1 de VMs: https://learn.microsoft.com/en-us/azure/virtual-machines/sizes/general-purpose/bv1-series
- Microsoft Azure - Serviços gratuitos de máquinas virtuais: https://azure.microsoft.com/free/virtual-machines/
- Flask: https://flask.palletsprojects.com/
- gunicorn: https://gunicorn.org/
- SQLite: https://www.sqlite.org/

A Código-fonte completo: app.py

app.py

```python
"""API de Tarefas - Flask + SQLite.

CRUD minimo de tarefas. O caminho do banco vem da variavel DB_PATH
(na VM da Azure o banco fica em /var/lib/api-tarefas/tarefas.db).
"""
import os
import sqlite3
from datetime import datetime, timezone

from flask import Flask, g, jsonify, request

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DB_PATH = os.environ.get("DB_PATH", os.path.join(BASE_DIR, "data", "tarefas.db"))

app = Flask(__name__)
app.json.ensure_ascii = False  # mantem acentos no JSON

SCHEMA = """
CREATE TABLE IF NOT EXISTS tarefas (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    titulo        TEXT    NOT NULL,
    descricao     TEXT    NOT NULL DEFAULT '',
    concluida     INTEGER NOT NULL DEFAULT 0,
    criada_em     TEXT    NOT NULL,
    atualizada_em TEXT    NOT NULL
);
"""


def agora():
    return datetime.now(timezone.utc).isoformat(timespec="seconds")


def conectar():
    conn = sqlite3.connect(DB_PATH, timeout=10)
    conn.row_factory = sqlite3.Row
    return conn


def init_db():
    os.makedirs(os.path.dirname(DB_PATH), exist_ok=True)
    conn = conectar()
    conn.execute("PRAGMA journal_mode=WAL")
    conn.executescript(SCHEMA)
    conn.commit()
    conn.close()


def get_db():
    if "db" not in g:
        g.db = conectar()
    return g.db


@app.teardown_appcontext
def fechar_db(_erro):
```

```
    db = g.pop("db", None)
    if db is not None:
        db.close()


def tarefa_para_dict(row):
    return {
        "id": row["id"],
        "titulo": row["titulo"],
        "descricao": row["descricao"],
        "concluida": bool(row["concluida"]),
        "criada_em": row["criada_em"],
        "atualizada_em": row["atualizada_em"],
    }


def buscar(tarefa_id):
    return get_db().execute(
        "SELECT * FROM tarefas WHERE id = ?", (tarefa_id,)
    ).fetchone()


def erro(mensagem, status):
    return jsonify({"erro": mensagem}), status


def validar(dados, parcial):
    """Retorna (campos_validos, mensagem_de_erro)."""
    if not isinstance(dados, dict):
        return None, "o corpo deve ser um objeto JSON"

    campos = {}

    if "titulo" in dados:
        titulo = dados["titulo"]
        if not isinstance(titulo, str) or not titulo.strip():
            return None, "'titulo' deve ser um texto nao vazio"
        if len(titulo.strip()) > 120:
            return None, "'titulo' deve ter no maximo 120 caracteres"
        campos["titulo"] = titulo.strip()
    else:
        if not parcial:
            return None, "'titulo' e obrigatorio"

    if "descricao" in dados:
        if not isinstance(dados["descricao"], str):
            return None, "'descricao' deve ser um texto"
        campos["descricao"] = dados["descricao"]

    if "concluida" in dados:
        if not isinstance(dados["concluida"], bool):
            return None, "'concluida' deve ser true ou false"
        campos["concluida"] = int(dados["concluida"])

    return campos, None


@app.get("/")
def raiz():
    return jsonify(
        {
            "api": "API de Tarefas",
            "versao": "1.0",
            "rotas": [
                "GET /health",
                "GET /tarefas",
                "GET /tarefas/<id>",
                "POST /tarefas",
                "PUT /tarefas/<id>",
                "DELETE /tarefas/<id>",
```

```
            ],
        }
    )


@app.get("/health")
def health():
    get_db().execute("SELECT 1").fetchone()
    return jsonify({"status": "ok", "banco": "sqlite"})


@app.get("/tarefas")
def listar():
    filtro = request.args.get("concluida")
    if filtro in ("0", "1"):
        rows = get_db().execute(
            "SELECT * FROM tarefas WHERE concluida = ? ORDER BY id", (int(filtro),)
        ).fetchall()
    else:
        rows = get_db().execute("SELECT * FROM tarefas ORDER BY id").fetchall()
    return jsonify([tarefa_para_dict(r) for r in rows])


@app.get("/tarefas/<int:tarefa_id>")
def obter(tarefa_id):
    row = buscar(tarefa_id)
    if row is None:
        return erro("tarefa nao encontrada", 404)
    return jsonify(tarefa_para_dict(row))


@app.post("/tarefas")
def criar():
    campos, msg = validar(request.get_json(silent=True), parcial=False)
    if msg is not None:
        return erro(msg, 400)

    db = get_db()
    ts = agora()
    cur = db.execute(
        "INSERT INTO tarefas (titulo, descricao, concluida, criada_em, atualizada_em)"
        " VALUES (?, ?, ?, ?, ?)",
        (
            campos["titulo"],
            campos.get("descricao", ""),
            campos.get("concluida", 0),
            ts,
            ts,
        ),
    )
    db.commit()
    return jsonify(tarefa_para_dict(buscar(cur.lastrowid))), 201


@app.put("/tarefas/<int:tarefa_id>")
def atualizar(tarefa_id):
    if buscar(tarefa_id) is None:
        return erro("tarefa nao encontrada", 404)

    campos, msg = validar(request.get_json(silent=True), parcial=True)
    if msg is not None:
        return erro(msg, 400)

    db = get_db()
    for coluna, valor in campos.items():
        # 'coluna' vem do dicionario validado acima, nunca direto do cliente
        db.execute(
            f"UPDATE tarefas SET {coluna} = ? WHERE id = ?", (valor, tarefa_id)
        )
    db.execute(
```

```
        "UPDATE tarefas SET atualizada_em = ? WHERE id = ?", (agora(), tarefa_id)
    )
    db.commit()
    return jsonify(tarefa_para_dict(buscar(tarefa_id)))


@app.delete("/tarefas/<int:tarefa_id>")
def remover(tarefa_id):
    db = get_db()
    cur = db.execute("DELETE FROM tarefas WHERE id = ?", (tarefa_id,))
    db.commit()
    if cur.rowcount == 0:
        return erro("tarefa nao encontrada", 404)
    return "", 204


@app.errorhandler(404)
def nao_encontrado(_e):
    return erro("rota nao encontrada", 404)


@app.errorhandler(405)
def metodo_invalido(_e):
    return erro("metodo nao permitido", 405)


init_db()

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000, debug=False)
```

B Arquivos de deploy

deploy/azure-cli.sh

```
#!/usr/bin/env bash
# Cria a infraestrutura na Azure (rode no Azure Cloud Shell ou com o az CLI logado).
set -euo pipefail

RG="rg-api-tarefas"
LOCATION="canadacentral"
VM="vm-gs"
SIZE="Standard_B2ats_v2"

# 0) (opcional) confira se o SKU esta disponivel na sua assinatura/regiao:
#    az vm list-skus -l "$LOCATION" --size "$SIZE" -o table
#    Se aparecer restricao, troque LOCATION (ex.: eastus2) ou SIZE.

# 1) Grupo de recursos
az group create --name "$RG" --location "$LOCATION"

# 2) VM Ubuntu 22.04 com disco de SO Premium SSD de 30 GiB e IP publico
az vm create \
  --resource-group "$RG" \
  --name "$VM" \
  --image Ubuntu2204 \
  --size "$SIZE" \
  --admin-username azureuser \
  --generate-ssh-keys \
  --storage-sku Premium_LRS \
  --os-disk-size-gb 30 \
  --public-ip-sku Standard

# 3) Libera a porta 80 (HTTP) no Network Security Group
az vm open-port --resource-group "$RG" --name "$VM" --port 80 --priority 900

# 4) IP publico da VM
az vm show -d --resource-group "$RG" --name "$VM" --query publicIps -o tsv

# Para apagar tudo depois da avaliacao (evita cobranca):
#   az group delete --name "$RG" --yes --no-wait
```

deploy/setup-vm.sh

```
#!/usr/bin/env bash
# Roda DENTRO da VM Ubuntu (via SSH), com o usuario azureuser.
# Uso: ./setup-vm.sh https://github.com/SEU_USUARIO/api-tarefas.git
set -euo pipefail

REPO_URL="${1:?Uso: ./setup-vm.sh <url-do-repositorio-git>}"
APP_DIR="/opt/api-tarefas"

sudo apt-get update -y
sudo apt-get install -y python3-venv git

sudo mkdir -p "$APP_DIR"
sudo chown "$USER":"$USER" "$APP_DIR"

```

```
if [ -d "$APP_DIR/.git" ]; then
    git -C "$APP_DIR" pull
else
    git clone "$REPO_URL" "$APP_DIR"
fi

python3 -m venv "$APP_DIR/.venv"
"$APP_DIR/.venv/bin/pip" install -r "$APP_DIR/requirements.txt"

sudo cp "$APP_DIR/deploy/api-tarefas.service" /etc/systemd/system/api-tarefas.service
sudo systemctl daemon-reload
sudo systemctl enable --now api-tarefas

sleep 2
sudo systemctl --no-pager status api-tarefas | head -n 8
echo
curl -s http://localhost/health
echo
```

deploy/api-tarefas.service

```
[Unit]
Description=API de Tarefas (Flask + SQLite via gunicorn)
After=network.target

[Service]
User=azureuser
WorkingDirectory=/opt/api-tarefas
Environment=DB_PATH=/var/lib/api-tarefas/tarefas.db
# cria /var/lib/api-tarefas com dono azureuser (onde fica o arquivo .db)
StateDirectory=api-tarefas
ExecStart=/opt/api-tarefas/.venv/bin/gunicorn --workers 2 --bind 0.0.0.0:80 app:app
# permite ouvir na porta 80 sem rodar como root
AmbientCapabilities=CAP_NET_BIND_SERVICE
NoNewPrivileges=true
PrivateTmp=true
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target
```